Ders 5 Genişletilmiş Notu: Algoritma Analizi, Asimptotik Gösterim ve İkili Arama

Ders 5 Genişletilmiş Notu: Algoritma Analizi, Asimptotik Gösterim ve İkili Arama

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Bir Algoritmanın Verimliliği Nasıl Ölçülür?
- 2. İşlem Sayma Örneği
- 3. Büyüme Oranı ve Asimptotik Analiz
- 4. Karmaşıklıkta Gözden Kaçırılmaması Gereken İki Durum
- 5. Algoritma Sınıfları
- 6. Temel İşlem: En Çok Zaman Alan İşe Odaklanmak
- 7. Sırasız Dizide Sıralı Arama
- 8. En Kötü, En İyi ve Ortalama Durum
- 9. İkili Arama
- 10. İkili Aramanın Karmaşıklığı
- 11. Soru: 0'ların Bitip 1'lerin Başladığı Yer

1. Bir Algoritmanın Verimliliği Nasıl Ölçülür?

Geçen hafta sıralama algoritmalarını görmüş, karmaşıklık hesabına da temel bir giriş yapmıştık. Hepsi n^2 karmaşıklıklı olan, ama birbirine göre avantajı ve dezavantajı bulunan algoritmalardı. Hoca bu hafta daha ilerlemeden önce karmaşıklık hesabının nasıl yapıldığını daha biçimsel bir şekilde toparlamak istediğini söyledi.

Başlangıç noktası şudur: Bir algoritma yazdığımızda dikkat etmemiz gereken ilk şey algoritmanın doğru olmasıdır. Doğru çalışan bir algoritmamız olduktan sonra önemli olan, zamanı ve yeri iyi kullanmasıdır. Geçen hafta karmaşıklıkları farklı iki algoritmanın, n büyüdüğünde çalışma sürelerinin geldiği noktayı görüp şaşırmıştık. Soru artık şudur: Bir algoritmanın zamanı iyi kullandığını nasıl ölçeceğiz?

1.1 Çalışma Süresini Doğrudan Ölçmek

İlk akla gelen yol, iki algoritmayı çalıştırıp süre tutmaktır. Çok büyük algoritmalarda bu bir çare olabilir. Hoca derin öğrenme yapan algoritmaları örnek verdi: Orada o kadar karmaşık işler olur ki karmaşıklığı hesaplamak gerçekten zordur. Bu durumda "şu donanımlı bir cihazda şu kadar milisaniye, saniye ya da dakika sürdü" diye süre verebilirsiniz. Ama hocanın sözüyle bu, çaresiz kalınan durumlar içindir. Hoca, çok karmaşık algoritmaların da karmaşıklığının bulunabildiğini, bunu abartmamak gerektiğini ayrıca ekledi.

Süre ölçülecekse doğru yöntem şudur: Asıl programın çalışmaya başladığı yerin hemen öncesinde bir zamanlayıcı (timer) başlatılır, bittiği yerde durdurulur. Böylece giriş/çıkış gibi algoritmanın çalışmasıyla ilgisi olmayan işler hesabın dışında kalır.

Daha temel durumlarda ise zaman ölçerek algoritmanın karmaşıklığı hakkında fikir yürütmek doğru değildir. Çünkü ölçülen süreyi algoritmadan başka pek çok şey etkiler.

1.2 Sisteme Bağlı Etkenler

Çalışma süresini etkileyen sisteme bağlı üç etken vardır:

- **Donanım:** İşlemci, bellek, bilginin önbellekte (cache) durup durmaması.
- **Yazılım:** Kullanılan dil ve araçlar. Java ile C arasında ciddi bir zaman farkı vardır, Assembly ile arasındaki fark daha da büyüktür. Derleyici (compiler) ya da yorumlayıcı (interpreter) kullanmak da fark yaratır. Bir programı C'de çalıştırmakla MATLAB ortamında çalıştırmak arasında "dağlar kadar" süre farkı vardır. Aynı dilde bile derleyiciye göre sonuç değişir: Bazı derleyiciler yazdığınız kodu optimize eder, bazıları etmez, bazılarında da bu seçenek açılıp kapatılabilir.
- **İşletim sistemi ve ortam:** İşletim sistemi, ağ ve o anda sistemde çalışan diğer uygulamalar programın hızını etkiler.

Dolayısıyla "şu donanımda, şu dille çalıştırdım" demek de yeterli değildir. Aynı işi yapan başka bir programla birebir aynı şartları sağlayıp karşılaştırmak hiç kolay değildir.

1.3 Sistemden Bağımsız Etkenler

Süreyi etkileyen iki etken daha vardır ve bunlar sisteme bağlı değildir. Birincisi algoritmanın kendisidir: Verimli bir algoritma daha kısa sürede çalışır. İkincisi giriştir. Giriş denince yalnız eleman sayısı kastedilmez. Elbette n'in 10 ya da 100 olması fark yaratır, ama girişin durumu da süreyi etkiler.

Soru: Girişin durumu çalışma süresini nasıl etkileyebilir? **Cevap:** Bir öğrenci verilerin rastgele girilebileceğini, çok büyük ve çok küçük sayıların bir arada, sırasız bulunabileceğini söyledi. Hoca en doğru noktanın sonuncusu olduğunu belirtti: Dizi sıralı, yarı sıralı ya da sırasız verildiğinde çalışma süresi farklı olur.

1.4 Objektif Ölçüm: n'e Bağlı Hesap

Aynı işi yapan iki programın zamanı ve yeri kullanımını objektif olarak karşılaştırmak için kullanılan yaklaşım, çalışma zamanını **girişteki eleman sayısı n'e bağlı olarak** söylemektir.

2. İşlem Sayma Örneği

Hoca, çalışma zamanının n'e bağlı olarak nasıl söylenebileceğini dizideki sıfırları sayan şu program parçası üzerinden gösterdi:

```
int count = 0;
int i, n;
for (i = 0; i < n; i++)
{
    if (a[i] == 0)
        count++;
}
```

Burada iki yol vardır. Birincisi her terimin zamanını, yani masrafını hesaplamaktır. Assembly'de `move`, `increment`, `add` gibi komutların kaçar çevrim (cycle) sürdüğünü görmüştük ve bunların birbirinden farklı olduğunu biliyoruz. Bir öğrencinin "ezberimizde değil" demesi üzerine hoca, bunun zahmetli olduğu için yapılmadığını söyledi. Tek tek "`if` kaç çevrim, atama kaç çevrim" diye hesap çıkarılmaz.

Yapılan şey, önce her işlemin **kaç defa yapıldığına** bakmaktır. Derste bu sayım öğrencilerle birlikte yapıldı:

İşlem	Kaç defa?
Değişken tanımı	3
Atama	2
< karşılaştırması	$n+1$
== karşılaştırması	n
Dizi erişimi	n

İşlem	Kaç defa?
Artırma	n ile $2n$ arası
Toplam	$4n+6$ ile $5n+6$ arası

Üç tane değişken tanımı vardır: `count`, `i`, `n`. İki tane atama vardır: `count = 0` ve `i = 0`.

`i < n` karşılaştırması $n+1$ defa yapılır. n defa döngüye girilir, $(n+1)$'inci kontrolde $i = n$ olur ve döngüden çıkılır. `a[i] == 0` karşılaştırması ve dizi erişimi ise döngünün içinde olduğu için n defa yapılır.

Artırma sayısında derste bir tartışma oldu. Öğrencilerden $2n$, $2n+1$ ve $n+1$ cevapları geldi. Hoca iki parçaya ayırıp çözdü:

- `a[i]` hep sıfırsa `count++` n defa çalışır. Hiç sıfır değilse hiç çalışmaz.
- `i++` her durumda n defa çalışır, $n+1$ defa değil. Hoca bunu küçük bir n ile gösterdi: $i = 0$ iken başlanır, bir artışta $i = 1$, ikinci artışta $i = 2$ olur. $n = 2$ için 2 artış, $n = 5$ için 5 artış vardır.

Dolayısıyla artırma sayısı en az n, en çok (worst case) $2n$'dir. Topladığımızda sayı $5+(n+1)+n+n+n = 4n+6$ ile $5+(n+1)+n+n+2n = 5n+6$ arasında kalır (ara adım eklendi). Bu hesapta atamanın ya da karşılaştırmanın süresinin birbirinden farklı olmasına hiç bakılmaz, yalnız kaç defa iş yapıldığına bakılır.

3. Büyüme Oranı ve Asimptotik Analiz

3.1 $5n+6$ mı, $5n+9$ mu?

Hoca şunu sordu: Biri algoritması için $5n+6$, başka biri $5n+9$ bulduysa hangisi daha iyidir? Bir öğrenci çok fark olmadığını söyledi, hoca da "çok doğru" dedi. Bir atamayı eksik ya da bir işi fazla yapmış olabilir. 3'lük bir fark önemli bir fark değildir.

Odaklanılması gereken şey, n **sonsuza giderken** ne olduğudur. Bir algoritmanın bu büyüme miktarına büyüme oranı (order of growth) denir. n çok büyüdüğünde sabitlerin önemi kalmaz.

3.2 Üç Kural

- **Katsayılar atılır.** $4n$ ile n arasında fark yoktur. n sonsuza giderken $2n$, $4n$ gibi bir farkın büyük önemi yoktur.
- **Sabitler atılır.** $4n+6$'daki 6'nın önemi yoktur. Bir iş ister 1 milyon defa çalışmış olsun, ister 1 milyon artı 6 defa, fark etmez.
- **En anlamlı terim alınır.** Karmaşıklık $3n^2+2n+5$ ise n'in önemi yoktur. n sonsuza giderken n'in n^2 yanında etkisi kalmaz.

Demek ki bir algoritmayı analiz ederken iki şey yapılır: Önce hesap girişteki eleman sayısı n'e bağlı olarak yapılır. Sonra sabitlerden ve küçük terimlerden kurtulup en anlamlı terim alınır. Hocanın örnekleri:

- $n+\log n$ çıktıysa n denir.
- n^2+n çıktıysa n^2 denir.
- $n\log n+n$ çıktıysa $n\log n$ denir.

3.3 Asimptotik Analiz ve Asimptotik Gösterim

Bir algoritmanın karmaşıklığının makineden, derleyiciden ve benzeri etkenlerden bağımsız olarak hesaplanmasına **asimptotik analiz (asymptotic analysis)** denir. Bunu göstermek için kullanılan notasyona da **asimptotik gösterim (asymptotic notation)** denir. Hoca, ne yaptığımızı zaten bilsek de terminolojiyi doğru kullanmanın önemli olduğunu söyledi.

Biz $2n^2+5n$ gibi bir şeye tak diye n^2 dedik, sabitleri ve katsayıları attık. Asimptotik gösterimler, bunun yanlış olmadığını, bu şekilde ifade edilebileceğini garanti eden notasyonlardır. Üç tanedir:

Gösterim	Sınır	Anlamı
Big-O, $O()$	Üst sınır (upper bound)	Algoritmanın karmaşıklığının olabileceği üst sınır
Big-Theta, $\Theta()$	Sıkı sınır (tight bound)	Algoritmanın karmaşıklığının kesin olarak ne olduğu
Big-Omega, $\Omega()$	Alt sınır (lower bound)	Algoritmanın en az ne olabileceği, altına inemeyeceği değer

Hocanın örnekleri sıralama algoritmalarındandı. Seçmeli sıralamanın (selection sort) karmaşıklığı en az n^2'dir, n karmaşıklıkla asla yapılamaz. Bu bir alt sınırdır. Eklemeli sıralamanın (insertion sort) alt sınırı en iyi durumda n, üst sınırı ise n^2'dir.

> **Ders sonrası not:** Hoca 00:28:25'te $n \log n + n$ ifadesinin n olarak gösterilebileceğini söyledi. Doğrusu $n \log n + n = O(n \log n)$'dir, çünkü en anlamlı terim $n \log n$'dir. Hoca aynı şeyi 00:25:11'de doğru söylemişti.

Bu gösterimlerin nasıl kullanıldığı önümüzdeki sene algoritma analizi dersinde ayrıntılı görülecek. Bu sene yalnız Big-O ile gidilecek: Karmaşıklığı $4n^2 + 5n$ bulduysanız n^2 diyeceksiniz.

> **Ek bilgi (derste anlatılmadı):** Bu üç gösterimin biçimsel tanımları şöyledir. $f(n) = O(g(n))$, her $n \geq n_0$ için $f(n) \leq c \cdot g(n)$ olacak şekilde pozitif c ve n_0 sabitlerinin bulunması demektir. $f(n) = \Omega(g(n))$ aynı koşulun $f(n) \geq c \cdot g(n)$ biçimidir. $f(n) = \Theta(g(n))$ ise ikisinin birlikte sağlanmasıdır.

3.4 Neden Hassas Hesaba Gerek Yok?

$4n^2 + 5n$ gibi bir ifade hassasiyet açısından iyi bir gösterim değildir. Birisi bir işi fazla yaptı diye algoritması daha mı kötü olacak? Ölçümü bu kadar hassas yapmaya gerek yoktur, çünkü n sonsuza giderken sabitlerin, katsayıların ve küçük terimlerin büyük önemi yoktur.

Hoca bunu iki algoritmayla gösterdi: $A = 100n + 1$ ve $B = n^2 + n + 1$ karmaşıklıklı olsun.

n	$A = 100n + 1$	$B = n^2 + n + 1$
10	$100 \cdot 10 + 1 = 1001$	$100 + 10 + 1 = 111$
100	$100 \cdot 100 + 1 = 10001$	$10000 + 100 + 1 = 10101$
10000	$100 \cdot 10000 + 1 = 1000001$	$100000000 + 10000 + 1 = 100010001$

$n = 10$ iken A çok daha büyüktür, $n = 100$ iken B onu geçmiştir, $n = 10000$ iken aralarında 100 kata yakın fark vardır. Hocanın anlatmak istediği şudur: n büyüdükçe n^2 çok etkili hâle gelir. n on milyon olduğunda, on milyonun karesinin üzerine eklediğiniz bir on milyonun büyük etkisi yoktur. Bu yüzden en yüksek anlamlı terim alınır.

> **Ders sonrası not:** Hoca $n = 10000$ için değerin 10^{10}'dan büyük olacağını söyledi. Doğrusu $B = 100\,010\,001$, yani yaklaşık 10^8'dir. Hocanın vardığı sonuç değişmez.

4. Karmaşıklıkta Gözden Kaçırılmaması Gereken İki Durum

4.1 $m \cdot n$ Karmaşıklığı

Hoca sordu: Karmaşıklığı $m \cdot n$ olan bir algoritmaya n denir mi? Bir öğrenci önce "evet" dedi. Başka biri m'in rastgele bir değer mi, girişe bağlı bir şey mi olduğunu sordu, m de n'e yakınsa n^2 diyeceğini söyledi. Hocanın cevabı, m'in durumunu gözden kaçırmamak gerektiğiydi.

Örneğin problem öğrenciler ve aldıkları derslerle ilgili olsun. 10 bin öğrenci varsa ve alınan ders sayısı 10 ise m'in büyük önemi yoktur. Ama problemin senaryosuna göre m de n gibi

değişebiliyor, büyüyebiliyorsa bu durum karmaşıklık hesabında göz önüne alınmalıdır. Bu durum iki şekilde gösterilir. Çoğunlukla $m \cdot n$ yazılır, böylece m küçükse çok önemli olmayacağı da anlaşılır. Nadiren de n^2 yazılır.

4.2 $2n^2 + cn$ Karmaşıklığı

İkinci soru bir c katsayısı içeren ifadeydi. Bir öğrencinin cevabı "doğrudan n^2" oldu, hoca da bunu doğruladı. Ama burada dikkat edilecek bir nokta vardır. c bir sabittir, fakat dışarıdan aldığımız bir bilgiye göre n'in katsayısı olarak belirleniyor olabilir. Küçük c değerleri için sorun yoktur. c örneğin 10^6 oluyorsa ne yapacaksınız? O katsayının çok büyük olma ihtimali varsa bunun farkında olmak gerekir.

Soru: $m \cdot n$ örneğinde m büyükse n^2 kabul edilebiliyordu. $c \cdot n$'deki c'yi de yüksek değerleri için n gibi kabul edip bu terimi n^2 alsak olmaz mı? **Cevap:** Olur. c'ye n desek terim n^2 olur, toplam yine n^2 kalır, karmaşıklık açısından bir şey değişmez. Hocanın işaret ettiği şey karmaşıklık hesabının sonucu değil, gerçek çalışma durumuydu. Asimptotik analizle "n sonsuza giderse ne olur" diye bakıyoruz, ama gerçekte normal dizilerle çalışıyoruz. n 1000 iken c 10^6 oluyorsa, karmaşıklık "n^2" olsa bile yazdığınız algoritmayı bu açıdan gözden geçirmeniz gerekir.

5. Algoritma Sınıfları

Hoca bunları ezberlemeye gerek olmadığını, kavramları doğru ifade edebilmenin önemli olduğunu söyledi.

Sabit zaman (constant time), $O(1)$**:** n'e bağlı bir durum yoksa karmaşıklığa 1 denir. Örnek, birinci sınıfta yapılan üçgen sorusudur: Üç kenar girilir, üçgenin eşkenar, ikizkenar ya da çeşitkenar olduğu bulunur. Arka arkaya karşılaştırmalar vardır ama hiçbiri n'e bağlı değildir. Üç sayının en küçüğünü bulmak da böyledir. İş 100 adım ya da 200 adım sürebilir, önemli değildir. 100 adımsa hep 100 adım sürer, hiçbir değişkene göre değişmez.

Logaritmik zaman (logarithmic time), $O(\log n)$**:** Örneğin `while (n > 1) n = n / 2;` gibi her seferinde ikiye bölerek giden algoritmalar. İkili arama (binary search) böyledir: 8 elemanınız varsa bir tur sonra 4, bir tur sonra 2, bir tur sonra 1 elemanınız olur.

Doğrusal zaman (linear time), $O(n)$**:** Örneğin bir dizinin en büyük elemanını bulmak. n defa iş yapılır.

$n \log n$ **zaman,** $O(n \log n)$**:** Dönemin sonuna doğru görülecek hızlı sıralama (quick sort), yığın sıralaması (heap sort) ve birleştirmeli sıralama (merge sort) böyledir. Bunlarda da bölerek iş yapmak söz konusudur.

Karesel zaman (quadratic time), $O(n^2)$**:** Genelde iç içe iki döngü (double loop) olan algoritmalar. Seçmeli sıralama, kabarcık sıralaması (bubble sort) ve iki boyutlu bilgiler üzerinde iç içe döngüyle matris işlemi yapmak bu sınıftadır.

Üstel zaman (exponential time), $O(2^n)$**:** Olası bütün durumların, bütün seçeneklerin değerlendirildiği algoritmalar. Hoca bunların gerçek zamanda çözülemeyen algoritmalar olduğunu ve algoritma analizi dersinin sonlarında görüleceğini söyledi.

> **Ders sonrası not:** Hoca üstel zamanlı algoritmaları "nondeterministik polinomsal algoritmalar" diye andı. Bu iki kavram aynı değildir. NP, evet cevabı polinom zamanda doğrulanabilen karar problemlerinin sınıfıdır. Dolayısıyla "NP" ile "üstel zaman" eş anlamlı değildir.

Soru: Bir de faktöriyel yok muydu? **Cevap:** Hoca, $n!$'in $n \cdot (n-1) \cdot (n-2) \cdots$ olduğunu, yani "n üzeri bir şey" sayılabileceğini söyledi. Faktöriyeli ayrı bir durum olarak değerlendirmenin doğru olmadığını, faktöriyel hesabının kendi başına bir algoritma olduğunu ve onun da bir karmaşıklığı bulunduğunu belirtti.

> **Ders sonrası not:** Hoca "faktöriyel zamanda çalışan algoritma diye bir şey yok" dedi. Bu doğru değildir. Burada iki ayrı şey karışmıştır: Faktöriyeli hesaplayan algoritma ile çalışma süresi $n!$ ile büyüyen algoritma. Bir dizinin bütün permütasyon-

larını tek tek deneyen algoritma $n!$ durum üretir, yani ikinci türdendir. $n \geq 4$ için $n! > 2^n$ olduğundan bu sınıf 2^n'den de hızlı büyür.

6. Temel İşlem: En Çok Zaman Alan İşe Odaklanmak

Molanın ardından hoca, $4n^2 + 2n + 5$ gibi bir sayıya komutları tek tek sayarak ulaşmanın ne kadar verimli olduğunu sordu. Cevabı şuydu: Çok verimli değil. Bir atama var, bir `if` var, bir `return` var diye tek tek uğraşmak, büyük bir algoritmada mantıklı değildir. Çünkü sonunda o katsayıları ve sabitleri zaten eleyeceksiniz.

Yapılması gereken, algoritmada **en çok zaman alan işi bulup yalnız onun karmaşıklığına odaklanmaktır**. n'e bağlı bir hesap yaptığımıza göre en çok zaman alan iş genelde döngülerdir. Birden çok döngü varsa en çok işlem yapanına bakılır. Hocanın örneği:

```
for (i = 0; i < n; i++)        /* iç içe iki döngü: n^2, buna odaklan */
    for (j = i; j < n; j++)
        /* ... */;

for (i = 0; i < n; i++)        /* tek döngü: n, bakmaya gerek yok */
    /* ... */;
```

İç içe döngüden n^2, tek döngüden n gelir. Bu, az önce konuştuğumuz $n^2 + n$ durumudur, yani ikinci döngüye bakmaya gerek yoktur.

7. Sırasız Dizide Sıralı Arama

Bu yaklaşımı en basit algoritmalardan biri üzerinde görelim: Elemanları sırasız bulunan bir dizide dışarıdan verilen bir elemanın yerini aramak. Dizi sırasız olduğu için başka çaremiz yoktur, elemanlara sırayla bakarız. Aranan 7 ise 7'yi bulana kadar gideriz. Dizide olmayan bir sayı, örneğin 11 aranıyorsa dizi bitene kadar gideriz.

x aranan sayı, `a` dizi, n dizinin uzunluğu olsun:

```
int sequentialSearch(int n, int a[], int x)
{
    int i = 0;
    while ((i < n) && (a[i] != x))
        i++;
    if (i < n)
        return i;
    else
        return -1;
}
```

> **Ders sonrası not:** Tahtadaki kodda `i` değişkeninin tanımı yoktu ve `i = 0;` yazılmıştı. Kodun derlenmesi için burada `int i = 0;` yazıldı.

Döngüde iki kontrol vardır. `i < n`, dizinin sonuna gelinip gelinmediğini kontrol eder, çünkü aranan eleman dizide yoksa dizi bitince döngüden çıkılmalıdır. `a[i] != x` ise aranan elemanın dizinin i'inci elemanına eşit olup olmadığını kontrol eder. İki şart sağlandığı sürece bir sonraki elemana geçilir.

Döngüden çıkmanın iki sebebi olabilir. `i < n` ise elemanı bulduğumuz için çıkmışızdır, indisi döndürürüz. `i == n` ise bütün elemanlara bakmış ama bulamamışızdır, bulunamadığını belirtmek için -1 döndürürüz.

Bu kodda en çok zaman alan iş döngüdür. Hesap, döngüdeki karşılaştırma sayısı üzerinden yapılır.

8. En Kötü, En İyi ve Ortalama Durum

8.1 En Kötü Durum (Worst Case)

En kötü durum, elemanın dizide olmadığı durumdur. Eleman dizide yoksa döngüye n defa girmek zorundasınız. Karşılaştırma sayısı için:

$$C_{worst}(n) = n$$

Yani bu, üst sınırdır.

8.2 En İyi Durum (Best Case)

En iyi durumda bir kere girip çıkılır, döngüde dönme diye bir şey yoktur. Baktığım ilk eleman aradığım elemandır. Hoca bu duruma çok takılmadığımızı söyledi, çünkü her zaman karşımıza çıkacak bir durum değildir.

8.3 Ortalama Durum (Average Case)

Gerçek dünyada en sık rastlanan, ortalama durumdur. Bu yüzden bazı tasarımlar masraf nedeniyle tamamen ortalama duruma göre yapılır. Hoca önceki haftalardan iki örneği hatırlattı. Deprem gibi bir olay olduğunda herkes aynı anda telefona sarılırsa kimse kimseye ulaşamaz. Üniversite sınavı sonuçları açıklandığında herkes aynı anda bağlanmaya kalkarsa sistem çöker. Bunun sebebi, sistemlerin "normalde aynı anda o kadar kişi hizmet istemeyecek" düşüncesiyle, ortalamaya uygun ve daha ucuz bir çözüm olarak kurulmasıdır.

Sıralı arama için ortalama durum hesabına bakalım. Aradığımız eleman dizide ya vardır ya yoktur.

Soru: Elemanın dizide bulunma olasılığı nedir? **Cevap:** Bir öğrenci %50 dedi. Hoca bunun "çok iddialı" olduğunu söyledi: Olasılık 0 ile 1 arasındadır. Bu olasılığa p diyoruz.

Soru: Eleman varsa birinci, ikinci ya da üçüncü sırada olma olasılığı nedir? **Cevap:** Önce $\frac{1}{n}$ cevabı geldi. Fakat elemanın dizide olmama ihtimali de olduğu için doğrusu $\frac{p}{n}$'dir. Olasılık bütün konumlar için aynıdır.

Soru: Bu durumda karşılaştırma sayısı kaçtır? **Cevap:** Eleman i'inci konumdaysa i defa karşılaştırma yapılır.

Hesapta iki parça vardır: Başarılı durum, yani elemanın dizide olması, ve başarısız durum, yani olmaması.

Başarılı durumda eleman birinci sıradaysa 1 karşılaştırma yapılır ve bunun olasılığı $\frac{p}{n}$'dir. İkinci sıradaysa 2 karşılaştırma yapılır, olasılık yine $\frac{p}{n}$'dir. En son n'inci sıradaysa n karşılaştırma yapılır. Başarısız durumun olasılığı $1-p$'dir ve n karşılaştırma yapılır. Toplarsak:

$$C_{avg}(n) = \left[1 \cdot \frac{p}{n} + 2 \cdot \frac{p}{n} + \cdots + n \cdot \frac{p}{n}\right] + n(1-p)$$

$\frac{p}{n}$ parantezine alınır ve $1 + 2 + \cdots + n = \frac{n(n+1)}{2}$ kullanılır:

$$C_{avg}(n) = \frac{p}{n}\left(1 + 2 + \cdots + n\right) + n(1-p) = \frac{p}{n} \cdot \frac{n(n+1)}{2} + n(1-p) = \frac{p(n+1)}{2} + n(1-p)$$

İki uç duruma bakalım:

- $p = 1$ ise eleman kesin dizidedir. İkinci terim sıfır olur, hesaba hiç girmez. Sonuç $\frac{n+1}{2}$'dir.
- $p = 0$ ise eleman kesin dizide yoktur. Birinci terim sıfır olur. Sonuç n'dir.

$\frac{n+1}{2}$'de katsayı ve sabit atılınca da n kalır. Demek ki ortalama durumda karmaşıklık n'dir.

Hocanın vurgusu: Ortalama durum, "ortalamasını aldım" demek değildir. Giriş ne küçükten büyüğe sıralıyken, ne büyükten küçüğe sıralıyken, ne aranan eleman en başta ya da en sondayken, rastgele bir giriş geldiğinde karmaşıklığın olasılıksal olarak nasıl hesaplandığıdır.

Ek bilgi (derste anlatılmadı): Hocanın burada kurduğu ifade, karşılaştırma sayısının beklenen değeridir: Her olası durumun karşılaştırma sayısı olasılığıyla çarpılıp toplanır. Yani yapılmayan şey, en iyi ve en kötü durum sayılarının düz ortalamasını almaktır. $p = 1$ için çıkan $\frac{n+1}{2}$'nin 1 ile n'in ortalamasına eşit olması, olasılıkların burada eşit dağılmasından gelir.

8.4 Neden En Kötü Duruma Odaklanıyoruz?

Ortalama durum hesabı kolay değildir. Gördüğünüz gibi sırasız bir dizide eleman ararken bile uğraştık. Bu yüzden algoritma kitaplarında ve iyi notlarda ortalama hesabı pek yapılmaz. İhtiyaç olarak da genellikle en kötü durumun bilinmesi önemlidir. Bu derste de, algoritma analizi dersinde de genellikle en kötü durum üzerinde durulacak.

Hocanın özetiyle bir algoritma yazdığınızda satır satır saymak yerine şunları yapacaksınız:

1. En çok zaman alan işi belirleyeceksiniz. Bu, n'e bağlı olarak döngülerdir.
2. Döngüler içinden en çok işlem yapanı bulacaksınız.
3. Hesabı en iyi, en kötü ve ortalama durum üzerinden yürütebilirsiniz. En çok odaklanmanız gereken en kötü durumdur.

Ortalama durumun özenle anlatıldığı bir yer hızlı sıralamadır (quick sort). En çok kullanılan sıralama algoritması olmasına rağmen en kötü durumu n^2'dir, dizi sıralıysa karmaşıklığı n^2 olur. Ortalama durumda ise $n \log n$'dir. Bu yüzden onun ortalama durumu değerlendirilecek.

Ders sonrası not: Sıralı dizide n^2 durumu pivot seçimine bağlıdır. Hoca bu ayrıntıya girmedi. Örneğin pivot olarak hep ilk eleman seçilirse sıralı dizide $\frac{n(n-1)}{2}$ karşılaştırma yapılır.

8.5 İki Toplam

Hoca, ortalama durum hesabındaki $\frac{n(n+1)}{2}$ için şunu hatırlattı:

$$\sum_{i=1}^{n} 1 = n, \qquad \sum_{i=1}^{n} i = \frac{n(n+1)}{2}$$

$i = 1$'den n'e kadar her seferinde bağımsız tek bir iş yapıyorsanız karmaşıklık n'dir. Her seferinde i defa iş yapıyorsanız, örneğin iç içe iki döngü durumunda, sonuç $\frac{n(n+1)}{2}$'dir ve karmaşıklık olarak n^2 değerlendirilir.

9. İkili Arama

Karmaşıklık hesabı için güzel örneklerden biri de ikili aramadır (binary search), çünkü $\log n$ karmaşıklığının nasıl hesaplandığını bir kere görmek gerekir. İkili arama, veri yapıları ve algoritmalarda hep söylediğimiz "verimiz belli bir düzende ise yapılan iş daha hızlı olabilir" fikrini kanıtlayan bir algoritmadır. Dizinin sıralı olması gerekir.

Mantık şudur: Dizi madem sıralı, önce orta göze bakılır. Aranan sayı ortadaki sayıdan büyükse, sol yarıdaki bütün sayılardan da büyüktür, oralara bakmaya hiç gerek yoktur. Küçükse de sağ yarı elenir. Yani bir seferde bir yarı elenir. Bu, azaltarak yönetme (decrease and conquer) algoritmasıdır ve ayrıntısı algoritma analizi dersinde görülecek. Gereksiz kısımları atarak ilerleriz.

Bunun kazancı büyüktür. 16 elemanınız varsa bir seferde 8'e, sonra 4'e, sonra 2'ye inersiniz. Hoca $2^{10} = 1024$ elemanlı dizi için doğrusal aramada (linear search) en kötü durumda 1024 karşılaştırma yapılacağını, ikili aramada ise işin çok daha az karşılaştırmayla çözüleceğini söyledi. Tam sayı için 10. bölümün sonundaki nota bakın.

9.1 Birinci Örnek: $x = 21$

Dizimiz sıralı 8 elemanlı bir dizi olsun, 21'i arayalım:

İndis	0	1	2	3	4	5	6	7
Değer	2	7	9	12	17	21	23	37

Baş	Son	Orta	Karşılaştırma	Sonuç
0	7	$(0+7)/2 = 3$	$21 > 12$	Sol yarı elenir, baş $= 3+1 = 4$
4	7	$(4+7)/2 = 5$	$21 = 21$	Eleman 5. gözde bulundu

21, 12'den büyükse solundaki hepsinden büyüktür. O hâlde baktığımız aralığın başlangıcı değişir ve ortanın bir ilerisinden başlar. Hoca "keşke 17'yi seçseymişim" dedi, çünkü 21 hemen ikinci adımda bulundu.

9.2 İkinci Örnek: $x = 13$, Eleman Dizide Yok

"Bakacak eleman kalmayana kadar devam etmek" ne demektir? Bunu görmek için dizide olmayan bir eleman arayalım:

İndis	0	1	2	3	4	5	6	7
Değer	7	11	22	35	41	56	71	82

Baş	Son	Orta	Karşılaştırma	Sonuç
0	7	3	$13 < 35$	Sağ taraf elenir, son $= 3-1 = 2$
0	2	1	$13 > 11$	Sol taraf elenir, baş $= 1+1 = 2$
2	2	2	$13 < 22$	Son $= 2-1 = 1$
2	1	-	Baş $>$ son	Durulur, eleman yok

13, 35'ten küçükse 41, 56, 71 ve 82'den de küçüktür, o tarafa bakmaya gerek yoktur. 11'den büyükse 7'den de büyüktür. 22'den küçükse sonu 22'den bir önceki göze çekeriz.

> **Ders sonrası not:** Hoca 01:51:09'da bir kere "13, 35'ten büyükse" dedi. Doğrusu "küçükse"dir. Hemen ardından da öyle devam etti.

Baş soldan sağa artarak, son sağdan sola azalarak gelir. Bir noktada son baştan küçük hâle gelir. Normalde başın sondan küçük ya da ona eşit olması gerekirken büyük olduğu an, bakılması gereken bütün bölgelerin işlendiği, dokunulmamış yer kalmadığı anlamına gelir. Dizi bitmiştir.

> **Hocanın vurgusu:** İşlem yaparken bitme durumunu dikkatli hesaplamak gerekir. Eleman dizide olmayabilir. Olmadığını anladığımız anda da işi kesmemiz gerekir.

9.3 Önce Eşitlik Kontrolü

Soru: Büyük mü diye baktık. Büyük değilse küçük ya da eşit olabilir. Son neden 3 değil de 2 oldu, üçe kadar bakmamız gerekmiyor muydu? **Cevap:** Hoca soruyu "çok güzel" buldu ve eksik bıraktığı yeri açıkladı. Aslında ilk yapılan şey `a[m] == x` kontrolüdür. Eşitse iş biter. Tahtadaki örnekte yalnız eşit olmama durumlarında hangi tarafa gidileceği gösterilmişti. Önce eşit mi diye bakılır, değilse hangi tarafta olduğu anlaşılmaya çalışılır. Orta göze zaten bakılmış olduğu için aralığa tekrar girmesi gerekmez.

9.4 Akış Diyagramı

Akış mı çizilsin, kod mu yazılsın sorusunda öğrencilerden kod isteyenler oldu. Hoca yine de akış diyagramını çizdi.

> **Hocanın vurgusu:** Tasarımlarınızı kodla değil, akış üzerinden yapın. Bu alışkanlığı edinirseniz çok rahat edersiniz, hatanız olursa çok çabuk düzeltirsiniz.

```
start
  -> oku: n, a[n], x
  -> l = 0, r = n - 1
  -> found = 0
  -> (l <= r) && (!found) ?
       T: m = (l + r) / 2
          a[m] == x ?
            T: found = 1
            F: x < a[m] ?
                 T: r = m - 1
                 F: l = m + 1
          -> döngü koşuluna geri dön
       F: found ?
            T: yaz: m
            F: yaz: "yok"
          -> stop
```

Akıştaki önemli noktalar şunlardır:

- `left <= right` koşulundaki eşitlik atlanmamalıdır. Aranan eleman `left` ile `right` birbirine eşitken de karşınıza çıkabilir. Hoca buna "durma şartı" dedi. Akışta bu koşul döngünün devam koşuludur: `found` sıfır olduğu ve koşul sağlandığı sürece dönülür.
- x, `a[m]`'den küçükse `r = m - 1`, değilse `l = m + 1` yapılır. Hocanın 3, 5, 7 dizisiyle anlatımı: $x = 5$ ise 7'den küçüktür, o taraf elenir.
- Döngüden çıkınca `found` kontrol edilir. Bulunduysa `m` yazdırılır, bulunmadıysa "yok" yazdırılır.

> **Hocanın vurgusu:** Değişmeyen hata, `right` ya da `left` değerini `mid` yapmaktır. Bu kesinlikle doğru değildir, çünkü o elemana zaten baktınız ve orada işiniz bitti.

> **Ders sonrası not:** Altyazıda 01:58:11'de "`left`'i `mid - 1` yapacaksınız" geçiyor. Doğrusu `left = mid + 1`'dir. Bu büyük olasılıkla dil sürçmesi ya da altyazı hatasıdır, hoca aynı kuralı 01:50:01'de "orta artı bir" diye doğru söylemişti.

Soru: İlk `if` kutusunun `True` kolu bağlanmamış, değil mi? **Cevap:** Hoca haklı olduğunu söyleyip teşekkür etti ve `True` kolunu akışa bağladı. Yukarıdaki akışta bağlantı eklenmiş hâliyle verilmiştir.

> **Hocanın vurgusu:** Bu çok temel ve mutlaka bilmeniz gereken algoritmalardan biridir. Çok farklı yerlerde, farklı şekillerde karşınıza çıkabilir.

Öğrenciler kodu akıştan kendilerinin yazabileceğini söylediği için kod derste yazılmadı.

> **Ek bilgi (derste anlatılmadı):** Akışın C karşılığı aşağıdadır, hocanın 21 ve 13 örnekleriyle sırasıyla 5 ve -1 döndürür. `right = mid` ya da `left = mid` yazılırsa aralık daralmayabilir ve döngü hiç bitmeyebilir. Bu, hocanın verdiği uyarının pratikteki sonucudur.

```c
int binarySearch(int n, int a[], int x)
{
    int l = 0, r = n - 1, m = 0, found = 0;
    while ((l <= r) && !found) {
        m = (l + r) / 2;
        if (a[m] == x)
            found = 1;
        else if (x < a[m])
            r = m - 1;
        else
```

```
            l = m + 1;
        }
        if (found)
            return m;
        return -1;
    }
```

10. İkili Aramanın Karmaşıklığı

En iyi durum 1'dir, bakar bakmaz bulduğumuz durumdur, üzerinde konuşmaya gerek yoktur. Biz en kötü duruma bakıyoruz: Eleman dizide yoktur ya da son adımda bulunur.

Hoca bunun anlaşılmasını özellikle önemsedi ve 5 elemanlı bir diziyle bir daha anlattı. $x = 13$ olsun. $5/2 = 2$, yani 2. göze baktık, orada 7 var. 13 ile 7'yi karşılaştırdım, bu bir karşılaştırmadır. Eşit olsa çıkıp gidecektim, ama eşit değil, 13 büyük. Geriye kalanın yarısını eleyeceğim ve önceki durumda yaptığımın yarısı kadar aynı işi yapacağım.

Yani n elemanlı durumdaki iş, bir karşılaştırma artı $n/2$ eleman için aynı iştir. Tek eleman kaldıysa ya vardır ya yoktur, bir karşılaştırma yapılır ve iş biter:

$$C_w(n) = C_w(n/2) + 1 \quad (n > 1), \qquad C_w(1) = 1$$

$C_w(n/2)$ için de aynı şey geçerlidir: $n/2$ eleman varken aradığımı bulamazsam onu da ikiye böleceğim, aynı işi $n/4$ için yapacağım.

Soru: $n/4$ olsaydı artı 2 mi yapacaktık? **Cevap:** Evet, aşağıdaki açılımda görüldüğü gibi $C_w(n/2) = C_w(n/4) + 1$ olduğundan önceki 1 ile birlikte artı 2 olur.

$$C_w(n) = C_w(n/2) + 1 = C_w(n/4) + 2 = C_w(n/8) + 3 = ...$$

Burada bir örüntü vardır: Sağdaki sayı i ise bölen 2^i'dir. $i = 2$ için $2^2 = 4$, $i = 3$ için $2^3 = 8$. Genel olarak:

$$C_w(n) = C_w(n/2^i) + i$$

Bu nereye kadar gider? Bizi bu n'li durumdan kurtaracak şey, parantezin içinin 1 olmasıdır, çünkü $C_w(1) = 1$'dir. $\frac{n}{2^i} = 1$ demek $n = 2^i$ demektir, bu da $i = \log_2 n$ demektir. Yerine koyarsak:

$$C_w(n) = C_w(1) + \log_2 n = \log_2 n + 1$$

Sabit atılınca $C_w(n) = O(\log_2 n)$ olur.

Hoca bunu bir kere daha kısaca özetledi. Yapılan işlemin adı **geriye doğru yerine koyma (backward substitution)**'dır ve algoritma analizi dersinde daha ayrıntılı görülecek. Her adımdaki durum, yarı boyuttaki durum artı bir karşılaştırmadır. Bunlar yazıldıkça örüntü görülür, genel ifade yazılır, sonra durma koşulu $n/2^i = 1$ yerine konur. Hoca, sonucun anlamını $2^{10} = 1024$ eleman üzerinden yine vurguladı: Burada $i = 10$ olur, adım sayısı $\log_2 n$ mertebesindedir.

> **Ders sonrası not:** Hoca 1024 elemanlı dizide en fazla 10 karşılaştırma yapılacağını söyledi. Bu, kendi bulduğu $\log_2 n+1$ formülüyle tam uyuşmaz. $n = 1024$ için en kötü durumda 11 elemana bakılır, $n = 16$ için 5, $n = 8$ için 4 (python ile sayıldı). 10 sayısı, yarıya bölme adımı sayısıdır. Son bakılan tek eleman da bir karşılaştırmadır. Üstelik $C_w(n/2)$ yazımı n'in 2'nin kuvveti olduğu durum içindir, genel sonuç $\lfloor \log_2 n \rfloor + 1$'dir. Büyüme oranı değişmez: $O(\log_2 n)$.
>
> **Ders sonrası not:** Hoca her adımı bir karşılaştırma saydı. Akışta eşit olmayan bir adımda `a[m] == x` ve `x < a[m]` olmak üzere iki karşılaştırma vardır. Bu yalnız sabit bir katsayı getirir, karmaşıklık yine $O(\log_2 n)$'dir.

11. Soru: 0'ların Bitip 1'lerin Başladığı Yer

> **Hocanın vurgusu:** Bu, hocanın sınavlarda sorduğu bir sorudur.

Elimizde önce 0'ların, sonra 1'lerin geldiği sıralı bir dizi var. Örneğin:

`0 0 0 1 1 1 1 1 1 1`

0'ların bitip 1'lerin başladığı yeri $\log n$ karmaşıklıkta nasıl buluruz?

Bir öğrenci ikili aramaya benzer bir mantık önerdi. Hoca sıfırların solda olduğunu, yani dizinin sıralı olduğunu doğruladıktan sonra başlangıç kısmını kendisi yazıp kontrolü öğrencilere bıraktı. Tartışma şöyle ilerledi:

Düz ikili arama tek başına yetmez. 10 elemanlı dizide 5. göze baktım, 1 buldum. Bu bana tek başına bir şey ifade etmez. Bir öğrenci 4. ve 6. gözlere de bakıp ikisi de 1 ise sola kaymayı önerdi. Hoca 6. göze bakmaya gerek olup olmadığını sordu. Gerek yoktur, 4. göze bakmak yeter. Sonra bir öğrenci önce 1'i bulup ardından ilk 0'ı aramayı önerdi. Hoca buna gerek olmadığını düşündürdü. Sonunda öğrenciler doğru fikre vardı: **1 buldukça sola, 0 buldukça sağa gitmek.**

Hoca bunu ikinci bir diziyle de gösterdi: `0 0 0 0 0 0 0 1 1`. $8/2 = 4$, ilk bakılan eleman 0'dır. Demek ki mantık şöyle kurulmalıdır:

- Ortadaki eleman 1 ise sola doğru bakılır.
- Ortadaki eleman 0 ise sağa doğru bakılır.

> **Ders sonrası not:** Hoca burada kısa yoldan $10/2 = 5$ dedi. Akıştaki formül 10 elemanlı dizide $(0+9)/2 = 4$ verir. Bu örneğin mantığını değiştirmez, çünkü verilen dizide 4. göz de 1'dir.

Geriye durma koşulu kalıyor.

Soru: Ne olunca duracağız? **Cevap:** Öğrencilerden `left` ile `right` eşit olunca ve aralarındaki fark 1 olunca önerileri geldi. Hoca ikisine de hayır dedi: Bakılan `mid` elemanının sağındaki ya da solundaki elemana göre karar verilir. Aranan şey 0'dan 1'e geçilen yerdir.

- Bakılan eleman 1 ise solundakinin durumu önemlidir. Solundaki 0 ise durulur.
- Bakılan eleman 0 ise sağındakinin durumu önemlidir. Sağındaki 1 ise durulur.

Soru: Algoritma ilk 1'in indisini mi döndürmeli? **Cevap:** Fark etmez, 0-1 geçişi neresiyse. Son 0'ın yeri de döndürülebilir, ilk 1'in yeri de.

Hoca akışın geri kalanını çizmedi. Sıcağı sıcağına kendiniz çizip çözmenizi önerdi. Bu hem ikili arama bilginizi oturtur, hem de benzer ama farklı bir algoritmayı çözmüş olursunuz.

> **Ek bilgi (derste anlatılmadı):** Hoca çözümü bıraktığı için önce kendiniz deneyin. Aşağıda hocanın kurduğu mantığın bir C karşılığı vardır, ilk 1'in indisini döndürür, dizide hiç 1 yoksa -1 döndürür. `m == 0` ve `m < n - 1` kontrolleri, dizi dışına erişmemek içindir. Yukarıdaki iki diziyle sırasıyla 3 ve 7 döndürür, her uzunluğu 200'e kadar olan bütün sıralı 0/1 dizileriyle de sınanmıştır.

```c
int ilkBir(int n, int a[])
{
    int l = 0, r = n - 1, m;
    while (l <= r) {
        m = (l + r) / 2;
        if (a[m] == 1) {
            if (m == 0 || a[m - 1] == 0)
                return m;
            r = m - 1;
        } else {
            if (m < n - 1 && a[m + 1] == 1)
                return m + 1;
            l = m + 1;
        }
    }
```

```
        return -1;
    }
```

Dersin Toparlaması

Bu derste bir algoritmanın verimliliğinin saatle değil, girişteki eleman sayısı n'e bağlı olarak ölçülmesi gerektiğini gördük. Süre ölçümü donanımdan, dilden, derleyiciden ve işletim sisteminden etkilenir. İşlem sayma $4n + 6$ gibi sonuçlar verir, ama n sonsuza giderken katsayıların, sabitlerin ve küçük terimlerin önemi kalmaz. Asimptotik analiz bu mantığı adlandırır, bu sene Big-O kullanılacak. Satır satır saymak yerine en çok zaman alan işe, yani genelde döngülere odaklanılır ve en kötü durum esas alınır.

Sıralı arama üzerinde en kötü, en iyi ve ortalama durum hesabını, ikili arama üzerinde de `left <= right` koşulunu, `mid - 1` ve `mid + 1` güncellemelerini ve geriye doğru yerine koyma ile $O(\log_2 n)$ sonucunu gördük. 0/1 dizisi sorusu, ikili aramayı benzer bir probleme uyarlamak için hocanın kendi kendinize çözmenizi önerdiği çalışmadır. Asimptotik gösterimlerin ve geriye doğru yerine koymanın ayrıntısı algoritma analizi dersinde, $n \log n$ sıralamalar ise dönem sonunda görülecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.